Skinning LWC components: what not to do

MSD VD2 LWC PROTOTYPE · TEAM REFERENCE · 26 OCTOBER 2026

Why these rules exist

Skin a shipped component only through what Salesforce documents, so the skin survives Salesforce's automatic upgrades. That means attributes, utility classes, styling hooks and our own `sep-` classes. Salesforce upgrades every org three times a year and promises only the documented parts of a component: *"We document changes to the features and behavior of components, but don't document changes to their internals."*

This applies to anyone skinning a `lightning-*` component in the MSD VD2 prototype, or reviewing someone else's skin. Everything below is a "don't". The positive procedure is the `component-from-design` skill.

1. Don't reach inside a component

Anything past the component's host element is out of bounds at every tier, because Salesforce can change it in any release.

NEVER	WHY	LINTER RULE
Set or read a private hook: `--slds-s-*` or `--_slds-*`	The SLDS Linter reserves them "for internal Salesforce use". Existing uses are debt (`_docs/teirs-adherance.md`); add no more	`no-slds-private-var`
Write a selector that names an SLDS class, e.g. a bare `.slds-page-header { … }`	Restyles every instance with no opt-out. Put an opt-in `sep-` class beside it instead	`no-slds-class-overrides`
Chain a descendant selector into a component's internal markup	The shadow DOM it targets is undocumented and changes between releases	`no-slds-class-overrides`
Set an undocumented or "reserved for internal use" attribute to unlock a hook	Not part of the component's API. Check the catalog's `properties` first	none
Script a class onto a component to fake a state (hover, filled)	The shadow root owns that state. Record it as a gap	none

A selector stops at the host: the tag (`lightning-button`), the tag narrowed by a reflected attribute (`[variant='neutral']`), or a `sep-` class.

2. Don't skip down the ladder

Stop at the first rung that reaches the property. Jumping to a lower rung because it is quicker is the most common mistake.

RUNG	LEVER	EXAMPLE
1	A documented attribute or `variant`	`variant="label-stacked"`
2	An SLDS utility class on the host	`slds-m-bottom_small`
3	A `--dxp-*` hook (Theme panel), for site branding	heading sizes

RUNG	LEVER	EXAMPLE
4	The component's own `--slds-c-*` hook	`--slds-c-inputtext-color-border`
5	A global `--slds-g-*` hook, scoped to the host	`--slds-g-color-border-error-1` on `lightning-grouped-combobox`
6	A real property on our own `sep-` class	`.sep-listing-chip { … }`
7	A documented `::part()`	`lightning-button::part(button)`

- **Don't expect a utility to reach inside.** On a `lightning-*` host it styles only the host's own box. Padding, border, radius and shadow inside the shadow root need a hook.
- **A utility only gets in through inheritance.** It works for type, such as `slds-text-heading_small` on a label, and for placement. It never closes a gap inside the component.

3. Don't hard-code values

Every value comes from a token, so a theme change still reaches it.

- **No raw hex, px, rem or font values.** Use `--slds-g-*` or `--sep-*` tokens. A raw value is allowed only as a `var()` fallback, e.g. `var(--slds-g-spacing-3, 0.75rem)`, or where no hook exists, with a comment saying so (`no-hardcoded-values-slds2`).
- **Every `--slds-g-*` reference carries a fallback,** and the fallback is SLDS's light default, not our Figma value (`no-slds-var-without-fallback`).
- **No `!important`.** Our stylesheets already win: SLDS rules are either layered, or unlayered and loaded before ours.
- **No `--lwc-*` tokens** as a primary value (`lwc-token-to-slds-hook`).
- **No SLDS 1 classes that SLDS 2 dropped.** Grep the shipped stylesheet first; `slds-page-header_record-home` is the known trap (`no-deprecated-classes-slds2`).
- **No relative `url()` in a stylesheet.** It passes dev and the build, then 404s in the built site. Use the absolute `/public/assets/…` path.

4. Don't put a skin in the wrong place

Each kind of skin has one home. Put it anywhere else and it either leaks to other components or can't be found at handover.

SCOPE	LIVES IN	SELECTOR
Every instance of one shipped component	`src/assets/css/site/components/<tag>.css` — one file per component (`button.css`)	The tag: `lightning-button`
An opt-in variant	That component's file in `site/components/`	A `sep-` class
A site-wide skin of an SLDS blueprint	`src/assets/css/site/blueprints/<name>.css`	A `sep-` class beside the SLDS one
Only inside one of our wrappers	That wrapper's own `.css`	The wrapper's class
A site-wide token or branding value	`src/assets/css/design-tokens.css`	`:root`

- **Never edit `src/assets/css/site.css`.** It is the bundle, generated from `src/assets/css/site/` by `npm run css` (and on every save under `npm run dev`).

- **Don't leave switched-off CSS as a commented block.** Delete it; git keeps the history.
- **Don't put component hooks in `design-tokens.css` at `:root`.** They belong on the component's tag.
- **Don't set a `--slds-g-*` global anywhere except `design-tokens.css`.** Re-pointing one on a single host (rung 5) is a commented, listed exception.
- **Don't build spacing into a `sep-` class.** The class says what the element *is*; where it sits stays a utility in the markup.
- **Open every skin block with a comment** naming the design source, the component, the states covered, and the states not covered and why.

---

5. Don't rebuild or fake it

When the skin can't reach a property, record the gap. Don't work around it.

- **Never rebuild a shipped control** (dropdown, modal, combobox, tabs, pill, toggle) because its skin falls short. That is out of bounds.
- **Never build a new custom component without asking first.** Building one is the user's decision, not the developer's.
- **Never fake a state** the design draws but no hook reaches. "Filled" and "hover" are the usual gaps.
- **Never remove SLDS.** Salesforce: *"Don't remove SLDS if you plan to use flexible layouts or any of the Lightning base components."*

There are two gates, and passing one doesn't pass the other:

GATE	ASKS	DECIDED BY
Upgrade gate	Does this rely only on what Salesforce documents?	The rules: no exceptions
Custom-component gate	Should we build a control the library doesn't ship?	The user, asked before any code

An approved new component is still built only from documented parts.

---

6. Blueprints: the extra don'ts

Use SLDS blueprint markup only for one of the 37 structures no shipped component renders, such as the page header, path, timeline or tiles. That is the approved way to build those, but it is our code to maintain: *"If SLDS updates the related blueprint, your component code isn't updated automatically."*

CHOOSING A BLUEPRINT

- **Don't hand-write a blueprint that a component already renders.** Check `/design-system` → Blueprints first. It lists only the 37 with no component behind them; the other 52 are rendered by a `lightning-*` component, so use that component.
- **Don't use a class without checking that it ships** in `slds2.cosmos.css`. Two classes are declared by their blueprint but missing from the bundled stylesheet: `slds-table_column` and `slds-setup-assistant__step`. SLDS 1 classes such as `slds-page-header_record-home` are gone.
- **Don't reach for a blueprint until you've ruled out every other option.** Salesforce: *"Before you create a component from a blueprint, make sure that there's no other way to meet your use case."*

RESTYLING A BLUEPRINT

- **Never override the SLDS class itself.** A bare `.slds-page-header { border: none }` changes every page header on the site. Put an opt-in `sep-` class beside it: `class="slds-page-header sep-page-header"`.

- **Reach for things in this order:** the blueprint's own public `--slds-c-*` hook, then a real property on the `sep-` class, then an SLDS utility in the markup for a one-off.
- **Style an element inside the blueprint through its own `sep-` class**, set in the markup beside the SLDS one. `.sep-page-header .slds-page-header__title` still names an SLDS class and is flagged by `no-slds-class-overrides`.
- **Don't build spacing into the `sep-` class.** Placement stays a utility in the markup. SLDS has no responsive spacing utilities, so a breakpoint margin becomes a `sep-` modifier. Use it instead of `slds-m-*`, never alongside: the utility carries `!important` and wins at every width.
- **Don't "fix" a blueprint that behaves oddly** without checking first. A completed path step hiding its label (`slds-is-complete` flips it with `rotateX`) is correct.

ADDING BEHAVIOUR

- **Don't treat an interactive blueprint as simple markup.** The blueprint gives *"HTML and CSS … but doesn't provide logic"*. Adding state, keyboard handling or accessibility makes it Tier 4, our code to maintain.
- **Never hand-roll a blueprint for a control that ships.** Dropdown, modal, combobox, tabs, pill and toggle all have a `lightning-*` component, and rebuilding one is out of bounds.
- **Don't give a structural wrapper (Tier 2B) its own stylesheet or state.** It takes `@api` properties and SLDS utilities only.

THE ONE SANCTIONED EXCEPTION: BADGES

Every badge on the site is `<span class="slds-badge …">` blueprint markup, not `lightning-badge`, by the user's decision. `lightning-badge` renders a bold full pill that doesn't match Cosmos, and a theme class on its host can't reach inside. Status classes come from `data/vacancyRules` (`statusThemeClass`, `statusTagClass`).

PAGE-HEADER HOOKS: RESOLVED 2026-10-24

The page header has **no public hooks**. Every hook its rule reads is `--slds-s-pageheader-*`, which the SLDS Linter reserves *"for internal Salesforce use"* (`no-slds-private-var`). So restyle it with real properties on the `sep-` class:

```
.sep-page-header { border: 0; box-shadow: none; padding: 0; }
```

Style an element inside it through its own `sep-` class in the markup, e.g. `class="slds-page-header__title sep-page-header__title"`, never `.sep-page-header .slds-page-header__title`. `rules-lwc-layout.md`, `site.css`, `jobs3.css` and the Blueprints tab generator have been corrected to match.

---

7. Don't hide the risks

Three levers work in this prototype but aren't production-safe. On 2026-10-20 we agreed to use them here only, and to list every use for the developer at handover.

LEVER	WHY IT'S A RISK	WHAT TO DO
`--slds-c-*` component hooks (rung 4)	A developer preview in SLDS 2: *"Don't implement functionality in production with these."*	Use, and list in the register
Re-pointing a `--slds-g-*` global on one host (rung 5)	*"Value assignment is reserved for theming tools."* The linter doesn't flag it	Comment each use; list in the register
Legacy `--sds-c-*` hooks	The beta namespace, unsupported. Some names were never published	Ask first; list in the register

- **Regenerate the register after any skin change:** `node .claude/skills/component-from-design/styling-register.mjs` writes `_docs/handover-styling-register.md`.

- **Lint every skin** with the SLDS Linter. The new block must raise no `no-slds-var-without-fallback`, `no-slds-class-overrides` or private-hook findings.
- **Don't call a skin done because it compiles.** Three hooks in this project compiled and did nothing. Read the computed value off the rendered element for each state, or report the skin as unverified.

Where it's written

In the repository:

- `.claude/rules/rules-skinning.md`: the ladder, the never list, where a skin lives
- `.claude/skills/component-from-design/SKILL.md`: selectors, values, hook status, linting
- `_docs/customisation-tiers.md`: the tiers and *Out of bounds*, with Salesforce's wording
- `.claude/rules/rules-lwc-layout.md`: *Restyling a shipped blueprint* (`sep-` classes)
- `_docs/handover-styling-register.md`: every prototype-only lever in use

Salesforce sources:

- SLDS Linter rule descriptions
- LWC guide: styling hooks
- LWC guide: CSS anti-patterns

Quotes from developer.salesforce.com were taken from archived copies (April–July 2026). Re-check the wording before quoting it outside the team.